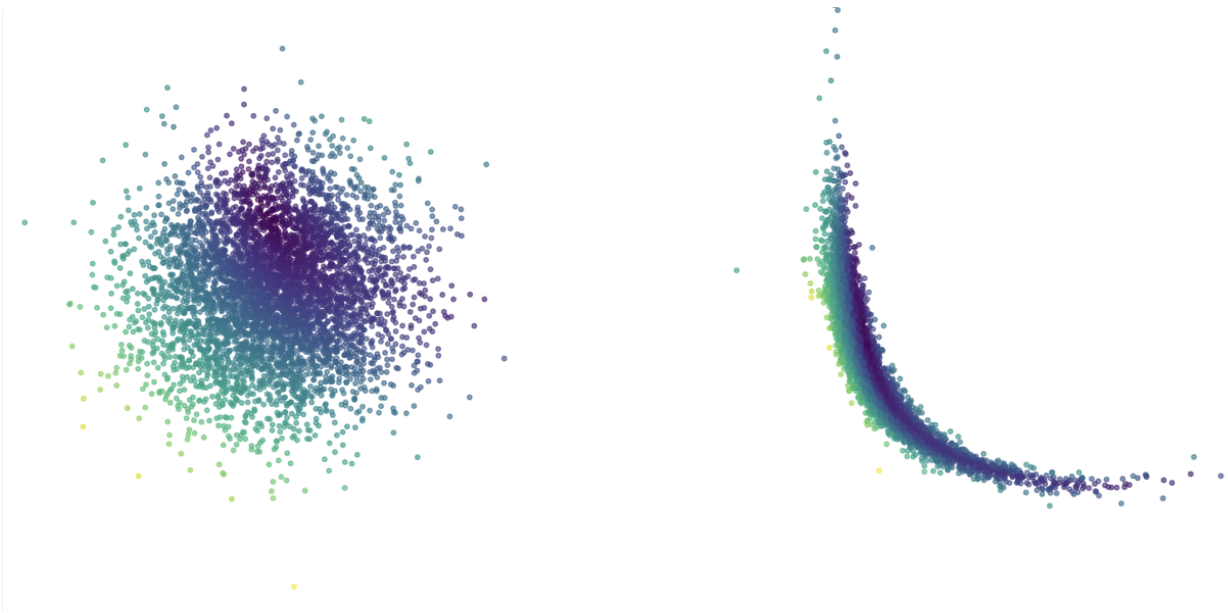
Sampling via Measure Transport

Picco Matteo
395305 – matteo.picco@epfl.ch

Raffo Luca
387774 – luca.raffo@epfl.ch



École polytechnique fédérale de Lausanne
Institute of Mathematics

December 2025

1 Introduction

In many applications we encounter the need to sample from complex probability distributions with a high computational cost associated with evaluating their density function, or presenting multi-modality, very strong correlations or many others.
A recent approach directed at alleviating this challenges is based upon the transport of measures. In this framework we start from a reference measure μ_{ref} that is easy to sample from and a target measure μ_{target} that presents some of the aforementioned issues. Then, we construct a transport map T from μ_{ref} to μ_{target}. In theory, to generate samples from μ_{target} we can just sample points from μ_{ref} and transform them using T. However, constructing a map T that *exactly* transforms μ_{ref} into μ_{target} is often out of reach, so we restrict ourselves to finding a good approximation of the map to improve other known sampling algorithms. We will apply these methods to a Bayesian inference for a biochemical oxygen demand problem.

2 Bayesian inference for a biochemical oxygen demand problem

We consider the following simplified model for the biochemical oxygen demand (BOD) of an aerobic organism:

$$\begin{aligned} B(t;x) &:= a(x_1)(1-e^{-b(x_2)t}),\ (x_1,x_2)=x\in\mathbb{R}^2 \\ a(x_1) &:= \left[0.4+0.4\left(1+\operatorname{erf}\left(\frac{x_1}{\sqrt{2}}\right)\right)\right] \\ b(x_2) &:= \left[0.01+0.15\left(1+\operatorname{erf}\left(\frac{x_2}{\sqrt{2}}\right)\right)\right], \end{aligned}$$

where $\operatorname{erf}(x)=\frac{1}{\sqrt{2}}\int_{-x}^{x}e^{-s^2}\mathrm{d}s$ is the error function, $x\in\mathbb{R}^2$ is an unknown random parameter and t represents time.

Suppose that at 5 different times $t_1=1,\dots,t_5=5$ we collect measurements $y_1,\dots,y_5$, where

$$y_i=B(t_i,x)+\varepsilon_i,\ \varepsilon_i\sim\mathcal{N}(0,\sigma^2),\ i=1,\dots,5.$$

We want to characterize the distribution of the set of random parameters $x\in\mathbb{R}^2$ conditioned on y (the measure data), which we will denote by $\pi(x|y)$. We will do it by sampling from it.

Assuming the independence between x and ε, by Bayes' theorem we have

$$\pi^y(x)=\pi(x|y)\sim\exp\left(-\frac{1}{2\sigma^2}\sum_{i=1}^{5}(y_i-B(t_i;x))^2\right)\eta(x), \tag{1}$$

up to a normalization constant, where we assume $\eta\sim\mathcal{N}(0,I_{2\times2})$, $\sigma^2=10^{-3}$ and

$$y=[0.18,0.32,0.42,0.49,0.54]. \tag{2}$$

2.1 Random Walk Metropolis

The first approach we use to sample from π^y is the Random Walk Metropolis, that is a version of the classic Metropolis-Hastings algorithm where the proposal density has zero mean and σ as a scaling parameter (also called the step size).

In particular, our proposal density will be $q(x,y)=g_\sigma(y-x)$, i.e. a bivariate Gaussian centered at the current state and with variance σ. In this case, the acceptance probability takes the form

$$\alpha(x,y)=\min\left\{\frac{f(y)}{f(x)},1\right\},$$

where f is the density of the target distribution π^y, which is known only up to a multiplicative constant, but that does not matter when we compute the acceptance probability.

Now we introduce the Random Walk Metropolis algorithm.

Algorithm 1 Random Walk Metropolis

Require: X_0
  for $n = 0, 1, \ldots, N$ **do**
    Generate $Y_{n+1} \sim q(X_n, \cdot)$ and $U \sim \mathcal{U}(0, 1)$
    if $U \leq \alpha(X_n, Y_{n+1})$ **then**
      Set $X_{n+1} = Y_{n+1}$
    else
      Set $X_{n+1} = X_n$

The Markov transition kernel of this algorithm is

$$P(x, A) = \int_A \alpha(x, z) q(x, z) \mathrm{d}z + (1 - \alpha^*(x)) \mathbb{1}_A(x)$$

where $\alpha^*(x) = \int \alpha(x, z) q(x, z) \mathrm{d}z$.

By the fact that $\alpha(x, z) q(x, z) f(z) = \alpha(z, x) q(z, x) f(x)$ it is easy to see that $\mathcal{P}$ is in detailed balance with the density of π^y, so that it is an invariant distribution for this transition kernel. Furthermore, it was proved in [JH00] that, denoting by $\pi^{n,\lambda}$ the law of the chain generated by (λ, P) at the n-th step, then $\pi^{n,\lambda} \to \pi^y$ with respect to the total variation norm. This means that this is a good algorithm to approximate samples from the distribution π^y.

This approach provides a well consolidated way of sampling from the posterior distribution, with the following diagnostics[1] (our chain runs 25000 steps, and we considered a burn-in period of 1000 steps):

AR	ESS	$\mathbb{E}_{mcmc}[x_1]$	$\mathbb{E}_{mcmc}[x_2]$
0.433	0.006	0.062	0.935

Table 1: Posterior sampling diagnostics for Random Walk Metropolis (after burn-in).

We show below how the autocorrelation decreases as we increase the lag.

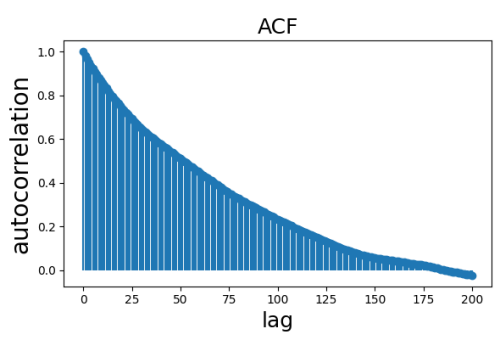
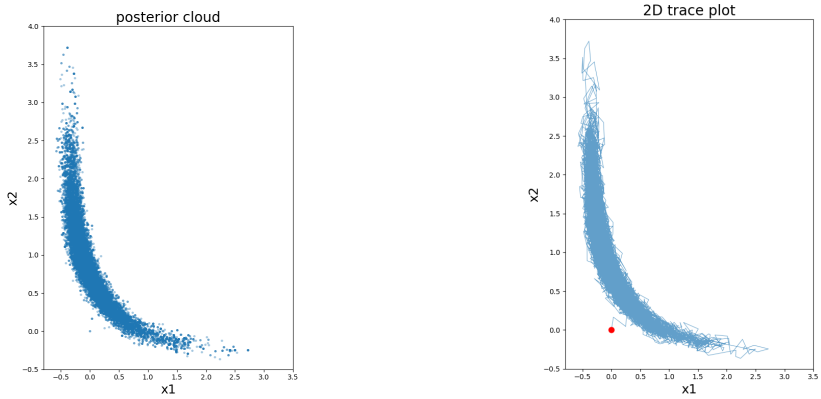


Figure 1: Autocorrelation decreases as the lag augments.

[1] AR stands for acceptance rate, and ESS stands for effective sample size.

Finally, we plot the posterior cloud of samples and the trace plot.



(a) Posterior cloud of samples.

(b) Trace plot of the Markov chain.

Figure 2: Posterior samples and trace diagnostics.

2.2 Parametric Triangular Maps

Another way to sample from the posterior distribution is to make use of the prior and of suitable transport maps. Indeed, if we were able to find a map T such that $(T)_{\#}\eta = \pi^y$, then we could sample $X \sim \eta$ (which is relatively easy) and notice that $T(X) \sim \pi^y$. However, finding an exact pushforward map may be difficult, so we can instead look for a transport map such that $(T)_{\#}\eta \approx \pi^y$ and use these samples in a Metropolis-Hastings algorithm (which is what we are going to do in the next section).

There can be infinitely many transformations that can approximate π^y well, but we will only consider the family of *parametric triangular maps*, as defined in [MMPS16]. In particular, denoting by T_{α_d} a general parametric map, the optimal map is obtained by solving the minimization problem

$$\arg\min_{\alpha_d} \ \mathcal{D}_{KL}^M((T_{\alpha_d})_{\#}\eta \| \pi^y), \tag{3}$$

where $\mathcal{D}_{KL}$ denotes the Kullback-Leibler divergence and $\mathcal{D}_{KL}^M$ its approximation

$$\mathcal{D}_{KL}^M(P\|Q) := \frac{1}{M}\sum_{i=1}^{M} \log\left[\frac{p(X_i)}{q(X_i)}\right], \quad X_i \overset{i.i.d.}{\sim} p,$$

where $P \sim p$ and $Q \sim q$. Since it will be useful to compute the optimal parametric map, we introduce the following characterization of (3).

Theorem 2.1. *If $X^{(i)} \overset{iid}{\sim} \mu$, and μ and ν have densities f and g with respect to the Lebesgue measure, and $g = \frac{1}{Z}\tilde{g}$ with $Z = \int_{\mathbb{R}^n} \tilde{g}(y)\, d\mathcal{L}^n$, then*

$$\begin{aligned}\arg\min_{T\in\mathbb{R}^d} D_{KL}^M(T\#\mu\|\nu) &= \arg\min_{T\in\mathbb{R}^d} D_{KL}^M(\mu\|T^{-1}\#\nu) \\ &= \arg\min_{T\in\mathbb{R}^d} \frac{1}{M}\sum_{i=1}^{M}\Big[-\log(\tilde{g}(T(X^{(i)}))) - \log|\det \nabla T(X^{(i)})|\Big].\end{aligned}$$

Proof. First, notice that if $X^{(i)} \overset{\text{iid}}{\sim} \mu$ then $T(X^{(i)}) \overset{\text{iid}}{\sim} T\#\mu$ by definition of push forward measure; moreover let us call f_T the density of $T\#\mu$, which takes the form given by Lemma A.3.

Then we can write:

$$\mathcal{D}_{KL}^{M}(T\#\mu\|\nu) = \frac{1}{M}\sum_{i=1}^{N}\log\left(\frac{f_T(Y^{(i)})}{g(Y^{(i)})}\right) = \frac{1}{M}\sum_{i=1}^{N}\log\left(\frac{f(T^{-1}(Y^{(i)}))|\det\nabla T^{-1}(Y^{(i)})|}{g(Y^{(i)})}\right)$$

By the inverse function theorem $|\det\nabla T^{-1}(y)| = \frac{1}{|\det\nabla T(x)|}$, so that applying the change of variable $y = T(x)$ and denoting with $g_{T^{-1}}$ the density of $T^{-1}\#\nu$ with respect to the Lebesgue measure, we can rewrite the RHS of the last equation as:

$$\frac{1}{M}\sum_{i=1}^{M}\log\left(\frac{f(X^{(i)})}{g(T(X^{(i)}))|\det\nabla T(X^{(i)})|}\right) = \frac{1}{M}\sum_{i=1}^{N}\log\left(\frac{f(X^{(i)})}{g_{T^{-1}(X^{(i)})}}\right) = \mathcal{D}_{KL}^{M}(\mu\|T^{-1}\#\nu),$$

where we have used again Lemma A.3 in the form $g_{T^{-1}}(x) = g(T(x))|\det\nabla T(x)|$.

Finally, notice that

$$\begin{aligned}
&\arg\min_T \mathcal{D}_{KL}^{M}(\mu\|T^{-1}\#\nu) = \arg\min_T \frac{1}{M}\sum_{i=1}^{M}\log\left(\frac{f(X^{(i)})}{g(T(X^{(i)}))|\det\nabla T(X^{(i)})|}\right)\\
&= \arg\min_T \frac{1}{M}\sum_{i=1}^{M}\Big[\log(f(X^{(i)})) - \log(g(T(X^{(i)}))) - \log|\det\nabla T(X^{(i)})|\Big]\\
&= \arg\min_T \frac{1}{M}\sum_{i=1}^{M}\Big[-\log\left(\frac{\tilde{g}(T(X^{(i)}))}{Z}\right) - \log|\det\nabla T(X^{(i)})|\Big]\\
&= \arg\min_T \frac{1}{M}\sum_{i=1}^{M}\Big[-\log(\tilde{g}(T(X^{(i)}))) - \log|\det\nabla T(X^{(i)})|\Big]
\end{aligned}$$

as the T does not appear in neither $\log(f(X^{(i)}))$ and $\log(Z)$ and thus we can disregard them in the optimization process. □

We notice that in the previous characterization it suffices to know the posterior distribution π^y only up to a multiplicative constant.

These maps are particularly useful because they are always invertible, as it it is proved in the following Theorem.

Theorem 2.2. *The map T_{α_d} is invertible for any choice of parameters α_d.*

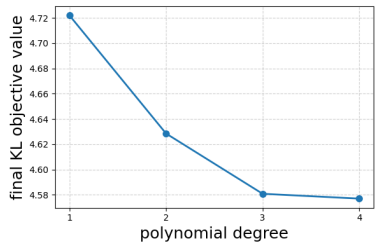
Proof. Notice that each component $T_{\alpha_d}^k$ only depends on $x_1, \ldots, x_k$, and therefore $J_{T_{\alpha_d}}$ is lower triangular. We know from linear algebra that a lower triangular matrix is invertible if and only if its diagonal elements are all different from zero. But from the definition of T_{α_d} and the fundamental theorem of calculus we know that:

$$\frac{\partial T^k}{\partial x_k} = \exp\left(\sum_{0\le i_1+\ldots+i_k\le d}\alpha_{k,i_1,\ldots,i_k}\cdot x_1^{i_1}\ldots{}_k^{i_k}\right) > 0$$

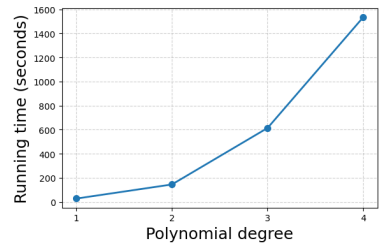
This implies that T is locally invertible around each point, so it is invertible everywhere. □

We ran the optimization problem (3) after fixing the polynomial degree of the transport map T. This procedure was repeated for increasing degrees $d = 1, 2, 3, 4$ in order to assess the impact of the map expressiveness in the approximation.

We show below the KL divergence between the transported samples from the prior (after having optimized the parameters) and the posterior samples, for each degree.



(a) Optimized KL divergence for different degrees.



(b) Time until convergence for each degree.

Figure 3: Optimization diagnostics for parametric maps of different degrees.

2.3 Improvements of the Markov Chain Monte Carlo algorithm

In this subsection, we will implement two different methods to improve the MCMC algorithm which make use of the optimal map T_{α_d} found in Subsection 2.2.

Independent Sampler. The first method involves the Independent Sampler algorithm (IS), where the proposal distribution is $(T_{\alpha_d})_{\#}\eta$, whose density we denote by g. Then the Transport Map Independent Sampler algorithm (TMIS) is the following:

Algorithm 2 Independence Sampler Metropolis–Hastings

Require: X_0
  for $n = 0, 1, \ldots, N$ **do**
    Generate $Y_{n+1} \sim g$ and $U \sim \mathcal{U}(0,1)$
    if $U \leq \tilde{\alpha}(X_n, Y_{n+1})$ **then**
      Set $X_{n+1} = Y_{n+1}$
    else
      Set $X_{n+1} = X_n$

In particular

$$\tilde{\alpha}(x,z) = \min\left\{\frac{f(z)}{f(x)}\frac{g(x)}{g(z)}, 1\right\},$$

where f is the density of the target distribution π^y. As it was noticed in the previous subsections, π^y is known only up to a normalizing constant, but to compute α we do not need to know it since, if $\tilde{f} = Cf$, then $f(Y_{n+1})/f(X_n) = \tilde{f}(Y_{n+1})/\tilde{f}(X_n)$, so we can use (1) to compute $\tilde{\alpha}$. Furthermore, to generate a random variable $Y \sim g$ we make use of the push-forward maps T_{α_d}: we generate $Z \sim \eta$ (which is relatively easy to do since η is Gaussian) and we set $Y = T_{\alpha_d}(Z)$.

The Markov transition Kernel of this algorithm is

$$P(x, A) = \int_A \tilde{\alpha}(x,z)g(z)\mathrm{d}z + (1 - \tilde{\alpha}^*(x))\mathbb{1}_A(x),$$

where $\tilde{\alpha}^*(x) = \int \tilde{\alpha}(x,z)g(z)\mathrm{d}z$. It is immediate to see that $\tilde{\alpha}(x,z)g(z)f(x) = \tilde{\alpha}(z,x)g(x)f(z)$, so that that (π^y, P) is in detailed balance, so that π^y is an invariant distribution for P.
In order to use the algorithm, we still need to derive a formula for g. In order to do this we can use Lemma A.3, so that we get

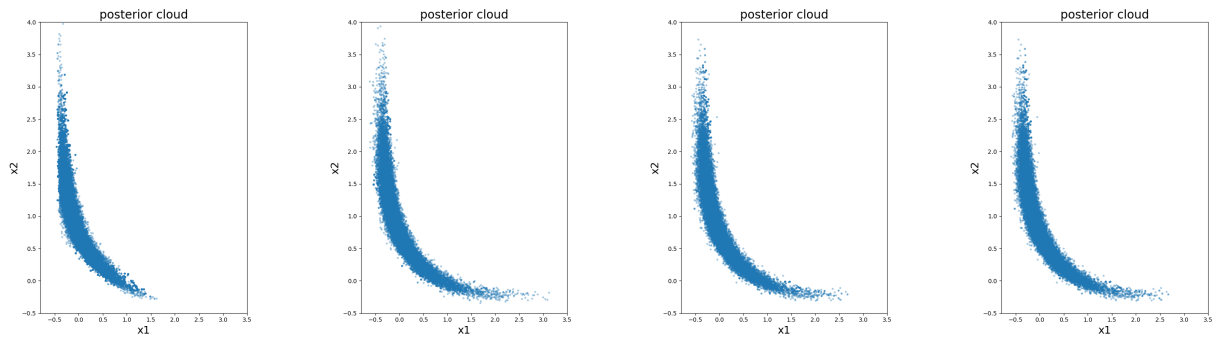
$$g(z) = \eta(T_{\alpha_d}^{-1}(z)) \det \nabla T_{\alpha_d}^{-1}(z) = \frac{\eta}{\det \nabla T_{\alpha_d}} \circ T_{\alpha_d}^{-1}(z).$$

We show below the diagnostics for different polynomial degrees.

Degree	AR	ESS	$\mathbb{E}_{mcmc}[x_1]$	$\mathbb{E}_{mcmc}[x_2]$
1	0.804	0.244	0.014	0.936
2	0.867	0.395	0.028	0.941
3	0.936	0.762	0.038	0.933
4	0.941	0.762	0.038	0.934

Table 2: Posterior sampling diagnostics and posterior expectations for independence sampler.

Below we plot the posterior cloud of samples for every polynomial degree.

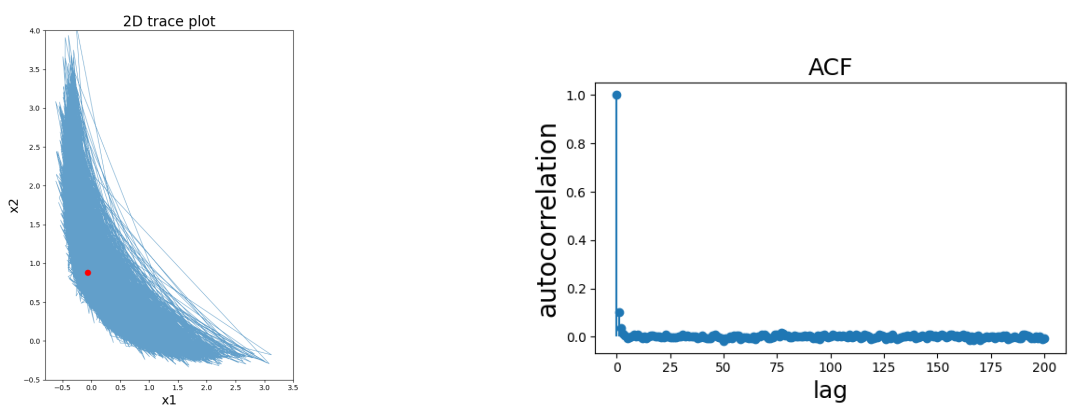


(a) $d = 1$. (b) $d = 2$. (c) $d = 3$. (d) $d = 4$.

Figure 4: Posterior cloud of samples for different degrees.

We can appreciate the fact that with a polynomial of degree one we never touch the right tail of our distribution, in fact we will use below a different approach with an Independence Sampler mixed with a Random Walk Metropolis which will guarantee convergence to the target distribution.

Finally, we plot for $d = 3$ the autocorrelation and the traceplot.



(a) Traceplot for TMIS, $d = 3$.

(b) Autocorrelation of TMIS, $d = 3$.

Figure 5: Traceplot and autocorrelation for TMIS, $d = 3$.

From this traceplot it is clear that the proposal is independent of the current state of the chain, as we notice some big jumps, and we can see that autocorrelation decays much faster.

TMIS and RWM. The second method uses a combination of the Transport Map Independent Sampler algorithm and the Random Walk Metropolis algorithm.

In particular, given $\gamma \in (0,1)$, at each iteration we do a step of RWM with probability γ or a step of TMIS with probability $1-\gamma$. The new algorithm is the following:

Algorithm 3 TMIS and RWM combined

Require: X_0
  for $n = 0, 1, \ldots, N$ **do**
    Generate $V \sim \mathcal{U}(0,1)$
    if $V \leq \gamma$ **then**
      Generate $Y_{n+1} \sim q(X_n, \cdot)$ and $U \sim \mathcal{U}(0,1)$
      if $U \leq \alpha(X_n, Y_{n+1})$ **then**
        Set $X_{n+1} = Y_{n+1}$
      else
        Set $X_{n+1} = X_n$
      end if
    else if $V > 1-\gamma$ **then**
      Generate $Y_{n+1} \sim g$ and $U \sim \mathcal{U}(0,1)$
      if $U \leq \tilde{\alpha}(X_n, Y_{n+1})$ **then**
        Set $X_{n+1} = Y_{n+1}$
      else
        Set $X_{n+1} = X_n$

We remark that for the steps that involve the Random Walk Metropolis algorithm we used the same notation as in Section 2.1.

It is immediate to see that the Markov transition kernel for this algorithm is

$$P(x, A) = \int_A (\gamma\alpha(x,z)q(x,z) + (1-\gamma)\tilde{\alpha}(x,z)g(z))\mathrm{d}z + (1 - \gamma\alpha^*(x) - (1-\gamma)\tilde{\alpha}^*(x))\mathbb{1}_A(x)$$

and that (f, P) is in detailed balance, so that π^y is an invariant probability measure for P.

The fact that the samples generated using the Random Walk Metropolis algorithm converge to the target distribution implies that also Algorithm 3 generates samples that converge to the posterior distribution π^y.

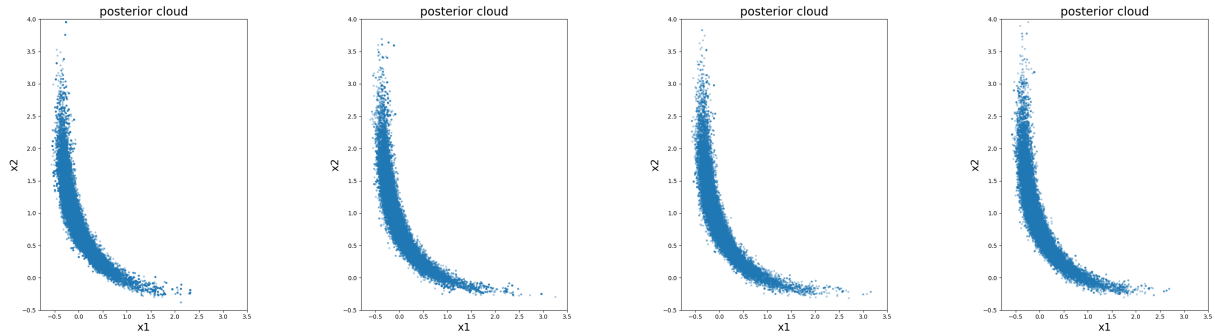
Mixing this algorithm with the Independent Sampler gives computational advantages. In our case we set $\gamma = 0.5$.

Degree	AR RWM	AR TMIS	ESS	$\mathbb{E}_{mcmc}[x_1]$	$\mathbb{E}_{mcmc}[x_2]$
1	0.428	0.773	0.043	0.046	0.930
2	0.427	0.859	0.125	0.046	0.930
3	0.427	0.938	0.296	0.044	0.927
4	0.426	0.942	0.297	0.041	0.933

Table 3: Posterior sampling diagnostics and posterior expectations for independence sampler mixed with random walk Metropolis with $\gamma = 0.5$.

We notice that the acceptance rates for both RWM and TMIS remain similar to those of the not-mixed algorithm. The effective sample sizes increase notably.

Below we plot the posterior cloud of samples for every polynomial degree.

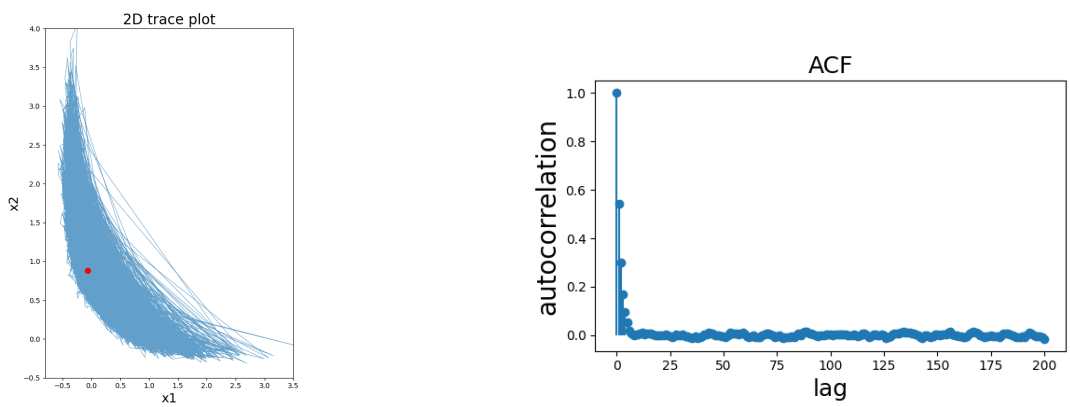


(a) $d = 1$. (b) $d = 2$. (c) $d = 3$. (d) $d = 4$.

Figure 6: Posterior cloud of samples for different degrees.

We can see the the posterior cloud for $d = 3$ is improved with respect to that of the Independence Sampler.

Finally, we plot for $d = 3$ the autocorrelation and the traceplot.



(a) Traceplot for the mixed algorithm, $d = 3$.

(b) Autocorrelation of the mixed algorithm, $d = 3$.

Figure 7: Traceplot and autocorrelation for the mixed algorithm, $d = 3$.

We can see here a mixed behavior between the non-mixed algorithms: an alternation between big and small jumps in the traceplot, and a quite fast decay in the autocorrelation.

2.4 Metropolis Hastings via the inverse of the optimal map

Another way of using push forward maps is by starting from the posterior distribution, so that instead of building the optimal parametric map T such that $(T)_{\#}\eta \approx \pi^y$, we construct a map S such that $(S)_{\#}\pi^y \approx \eta$.

More specifically, we use the same algorithm proposed in Section 2.2 but inverting the prior and the posterior distributions.

In order to do this, we need samples from π^y, which we can obtain by running a preliminary Random Walk Metropolis algorithm. Then, we use the map S to extend and improve our samples in the following way:

Algorithm 4 Metropolis-Hastings via the inverse map

Require: X_0
  for $n = 0, \dots, N$ **do**
    Compute $\hat{X}_n = S(X_n)$
    Generate $\hat{Y}_n \sim q(\hat{X}_n, \cdot)$
    Compute $Y_n = S^{-1}(\hat{Y}_n)$
    Generate $U \sim \mathcal{U}(0,1)$
    if $U \leq \beta(X_n, Y_n)$ **then**
      Set $X_{n+1} = Y_n$
    else
      Set $X_{n+1} = X_n$

We remark that $q(x, \cdot) \sim N(x, \sigma^2 I_{2\times 2})$. By β we denote the acceptance probability measure. Indeed, the new proposal distribution is $\tilde{q}(x, \cdot) \sim (S^{-1})_{\#} q(S(x), \cdot)$ which can be rewritten using Lemma A.3 as $\tilde{q}(x, z) = q(S(x), S(z)) \det \nabla S(z)$. This means that the new acceptance probability is

$$\beta(x, z) = \min\left\{\frac{f(z)}{f(x)}\frac{\tilde{q}(z, x)}{\tilde{q}(x, z)}, 1\right\}.$$

Given this acceptance probability, it is immediate to check that the Markov transition probability is

$$P(x, A) = \int_A \beta(x, z)\tilde{q}(x, z)\mathrm{d}z + (1 - \beta^*(x))\mathbb{1}_A(x),$$

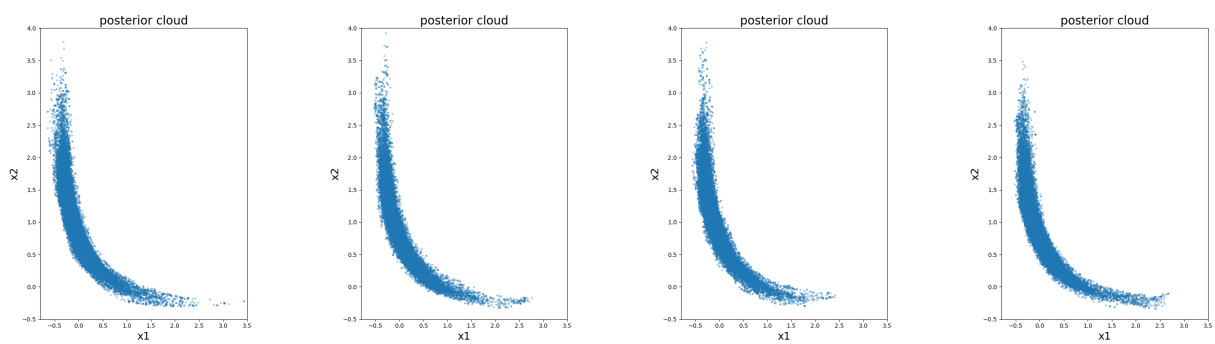
where $\beta^*(x) = \int \beta(x, z)\tilde{q}(x, z)\mathrm{d}z$. Again, we make use of the fact that $f(x)\tilde{q}(x, z)\beta(x, z) = f(z)\tilde{q}(z, x)\beta(z, x)$ to prove that (f, P) is in detailed balance, so that π^y is an invariant probability measure for P.

We show below the diagnostics for different polynomial degrees.

Degree	AR	ESS	$\mathbb{E}_{mcmc}[x_1]$	$\mathbb{E}_{mcmc}[x_2]$
1	0.887	0.004	0.087	0.938
2	0.904	0.009	0.065	0.884
3	0.904	0.009	0.011	0.976
4	0.909	0.010	0.059	0.908

Table 4: Posterior sampling diagnostics and posterior expectations for Metropolis-Hastings with the inverse map.

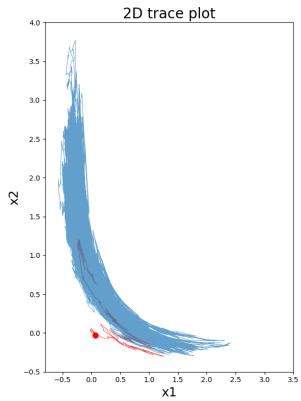
Below we plot the posterior cloud of samples for every polynomial degree.



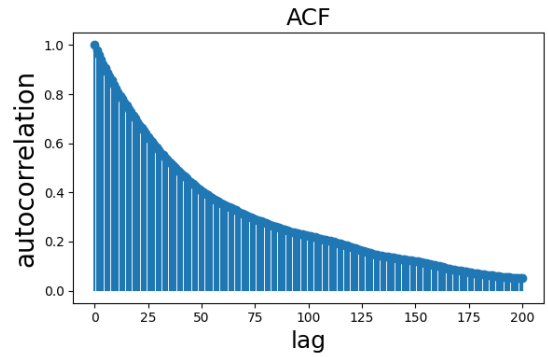
(a) $d = 1$. (b) $d = 2$. (c) $d = 3$. (d) $d = 4$.

Figure 8: Posterior cloud of samples for different degrees.

Finally, we plot for $d = 3$ the autocorrelation and the traceplot, in which we highlighted in red the first 200 steps of the chain to show its *horizontal* movements.



(a) Traceplot for the inverse algorithm, $d = 3$.



(b) Autocorrelation of the inverse algorithm, $d = 3$.

Figure 9: Traceplot and autocorrelation for the inverse algorithm, $d = 3$.

We notice that while running the previous algorithm we can actually improve the optimal map S. Indeed, as new samples of the posterior become available, we can restart the algorithm to find the optimal parametric map from π^y to η. However, due to computational issues, we might not be able to repeat this process at every iteration of the Metropolis-Hastings algorithm, but only when a significant new amount of samples becomes available. For this reason we compute the new map $S^{(n)}$ only every 500 iterations; furthermore, we initiate the minimization algorithm starting from the optimal parameter of the map $S^{(n-1)}$, which should make the process faster. We have the following:

Algorithm 5 Adaptive Metropolis-Hastings via the inverse map

Require: X_0
  for $n = 0, \dots, N$ **do**
    Generate $U \sim \mathcal{U}(0, 1)$
    if $n > 0$ and n is a multiple of 500 **then**
      Compute $S^{(n)}$ as parametric optimal map using the samples $(X_i)_{1 \le i \le n}$ and initiating the minimization from the previous optimal parameter α_d
      Compute $\hat{X}_n = S^{(n)}(X_n)$, generate $\hat{Y}_n \sim q(\hat{X}_n, \cdot)$ and set $Y_n = (S^{(n)})^{-1}(\hat{Y}_n)$
      if $U \le \beta^{(n)}(X_n, Y_n)$ **then**
        Set $X_{n+1} = Y_n$
      else
        Set $X_{n+1} = X_n$
    else
      Set $S^{(n)} = S^{(n-1)}$, compute $\hat{X}_n = S(X_n)$, generate $\hat{Y}_n \sim q(\hat{X}_n, \cdot)$ and set $Y_n = (S^{(n)})^{-1}(\hat{Y}_n)$
      if $U \le \beta^{(n)}(X_n, Y_n)$ **then**
        Set $X_{n+1} = Y_n$
      else
        Set $X_{n+1} = X_n$

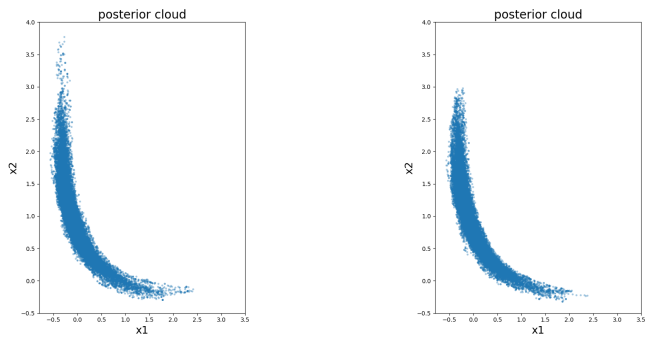
We notice that the acceptance probability changes at every iteration. Indeed, as we update the map $S^{(n)}$ the proposal distribution changes and therefore also the acceptance probability. In particular the proposal distribution has density

$$q_n(x, z) = q(S^{(n)}(x), S^{(n)}(z)) \det \nabla S^{(n)}(z)$$

so that

$$\beta^{(n)}(x,z) = \min\left\{\frac{f(z)}{f(x)}\frac{q_n(z,x)}{q_n(x,z)}, 1\right\}.$$

We show below the comparison between the algorithm with the inverse maps and the adaptive algorithm with the inverse map, for $d = 3$.



(a) Metropolis-Hastings with the inverse map, no adaptivity.

(b) Metropolis-Hasting with the inverse map with adaptivity.

Figure 10: Posterior cloud of samples for $d = 3$.

Appendix

In this section we recall some results and notions used throughout the report.

Lemma A.3. *If the probability measures μ_{ref} and μ_{target} admit corresponding densities f and g with respect to the Lebesgue measure, then*

$$T_{\#}\mu_{ref} = \mu_{target} \text{ if and only if } g = f \circ T^{-1}|\det \nabla T^{-1}|.$$

Definition A.4. *Let $P : \mathcal{X} \times \mathcal{B}(\mathcal{X}) \to [0,1]$ be a transition kernel and λ a probability measure on $\mathcal{X}$. We say that (λ, P) is in detailed balance if*

$$\int_B P(x,A)\mathrm{d}\lambda(x) = \int_A P(x,B)\mathrm{d}\lambda(x), \quad \forall\ A, B \in \mathcal{B}(\mathcal{X}).$$

Remark A.5. If (λ, P) is in detailed balance, then λ is an invariant measure for P.

References

[JH00] Søren Fiig Jarner and Ernst Hansen. Geometric ergodicity of metropolis algorithms. *Stochastic Processes and their Applications*, 85(2):341–361, 2000.

[MMPS16] Youssef Marzouk, Tarek Moselhy, Matthew Parno, and Alessio Spantini. *Sampling via Measure Transport: An Introduction*, pages 1–41. Springer International Publishing, 05 2016.

B Extracts of Code

```python
# observation times
t_obs = np.arange(1, 6)          # [1,2,3,4,5]

# observed noisy data
y_obs= np.array([0.18, 0.32, 0.42, 0.49, 0.54])

# noise variance sigma^2
sigma2 = 1e-3

def b_ox(x2):
    return 0.01 + 0.15 * (1.0 + erf(x2 / np.sqrt(2.0)))

def B_model(t, x):
    x1 = x[:, 0]
    x2 = x[:, 1]
    a = a_ox(x1)
    b = b_ox(x2)
    out = a[:, None] * (1.0 - np.exp(-b[:, None] * t[None, :]))
    if x.ndim == 1:
        return out[0]
    else:
        return out

def log_prior(x):
    lp = -0.5 * np.sum(x * x, axis = 1)
    if x.ndim == 1:
        return lp[0]
    else:
        return lp

def log_likelihood(x):
    pred = B_model(t_obs, x)
    if pred.ndim == 1:
        r = y_obs - pred
        return -0.5 * np.sum(r * r) / sigma2
    r = y_obs[None, :] - pred
    return -0.5 * np.sum(r * r, axis = 1) / sigma2

def log_posterior(x):
    return log_likelihood(x) + log_prior(x)
```

Listing 1: utilities

```python
def rwm(n_iter, step = 0.2, seed = 987):
  """ classic random walk metroplis to sample from the posterior distribution
      defined as pi^y """

  # random seed
  rng = np.random.default_rng(seed)

  # starting point
  x = np.zeros(2, dtype = float)

  # our chain
  chain = np.zeros((n_iter, 2))

  acc_count = 0

  for i in range(1, n_iter):

    # acceptance probability
    u = np.log(rng.uniform())

    # proposed step
    xp = x + step * rng.normal(size = 2)

    if u < (log_posterior(xp) - log_posterior(x)):
      x = xp
      acc_count += 1

    # update the chain
    chain[i] = x

  return chain, acc_count / n_iter
```

Listing 2: random walk metroplis

```python
def acf_2d(chain, max_lag = 2000):
  """ autocorrelation function """

  chain = np.asarray(chain, dtype=float)

  mean = chain.mean(axis=0)
  chain_centered = chain - mean

  # total variance (trace of covariance)
  var = np.mean(np.sum(chain_centered * chain_centered, axis=1))

  out = np.zeros(max_lag + 1)
  out[0] = 1.0

  # autocorrelation for each lag from 1 to max_lag
  for k in range(1, max_lag + 1):
    num = np.mean(np.sum(chain_centered[:-k] * chain_centered[k:], axis=1))
    out[k] = num / var

  return out
```

Listing 3: autocorrelation

```python
class TriangularTransport2D:
    def __init__(self, degree, n_quad = 20):
        """main definition of the object"""

        # we fix the polynomial degree and the quadrature infos
        self.d = degree
        self.xq, self.wq = np.polynomial.legendre.leggauss(n_quad)

        # we set the indices for the alpha_2 parameter (the others are trivial)
        self.alpha2_idx = [
            (i, j) for i in range(degree + 1)
                   for j in range(degree + 1 - i)
        ]

        # n_params gives the total number of parameters (#a1 + #alpha1 + #a2 + #alpha2)
        self.n_params = 1 + (degree + 1) + (degree + 1) + len(self.alpha2_idx)

    # ------------------------------------------------------------------

    def initial_theta(self, scale=0.01, seed=None):
        """ initialize each parameter with a random small number (using gaussian noise) """

        if seed is not None:
            np.random.seed(seed)

        theta = np.zeros(self.n_params)

        # n_params long cycle
        for i in range(self.n_params):
            theta[i] = scale * np.random.randn()

        return theta

    # ------------------------------------------------------------------

    def unpack(self, theta):
        """ starting from theta, we want to unpack the parameters a1, alpha1, a2, alpha2 """

        # degree of the polynomial
        d1 = self.d + 1

        #a1
        a1 = theta[0]

        #alpha1
        start = 1
        alpha1 = theta[start : start + d1]

        #a2
        start += d1
        a2 = theta[start : start + d1]

        #alpha2
        start += d1
        alpha2 = theta[start:]
```

```python

        # output
        return a1, alpha1, a2, alpha2

    # ----------------------------------------------------------------

    def p1(self, x, alpha1):
        """ first polynomial """

        val = 0.0
        for k in range(len(alpha1)):

            # add alpha1[k] * x^k term
            val += alpha1[k] * (x ** k)
        return val


    def p2(self, x1, x2, alpha2):
        """ second polynomial """

        val = 0.0
        for k in range(len(alpha2)):

            # get polynomial exponents
            i, j = self.alpha2_idx[k]

            # add alpha2[k] * x1^i * x2^j term
            val += alpha2[k] * (x1 ** i) * (x2 ** j)
        return val

    # ----------------------------------------------------------------

    def T1(self, x1, a1, alpha1):
        """ T^1 """

        if x1 == 0.0:
            return a1

        # map interval [0, x1] to [-1, 1]
        mid = 0.5 * x1
        half = 0.5 * x1

        s = 0.0
        for xi, wi in zip(self.xq, self.wq):

            # quadrature evaluation point
            t = mid + half * xi

            # accumulate exp(p1(t))
            # s += wi * np.exp(self.p1(t, alpha1))
            s += wi * self.safe_exp(self.p1(t, alpha1))

        return a1 + half * s


    def T2(self, x1, x2, a2, alpha2):
        """ T^2 """

        # polynomial part depending on x1
```

```python
        poly = 0.0
        for k in range(len(a2)):
            poly += a2[k] * (x1 ** k)

        if x2 == 0.0:
            return poly

        # map interval [0, x2] to [-1, 1]
        mid = 0.5 * x2
        half = 0.5 * x2

        s = 0.0
        for xi, wi in zip(self.xq, self.wq):

            # quadrature evaluation point
            t = mid + half * xi

            # accumulate exp(p2(x1, t))
            # s  += wi * np.exp(self.p2(x1, t, alpha2))
            s += wi * self.safe_exp(self.p2(x1, t, alpha2))


        return poly + half * s

    # ------------------------------------------------------------------

    def forward(self, x, theta):
        """ push forward the samples x """

        # get parameters
        a1, alpha1, a2, alpha2 = self.unpack(theta)

        n = len(x)
        y = np.zeros((n, 2))

        # push the sampls
        for i in range(n):
            x1 = x[i][0]
            x2 = x[i][1]

            y[i][0] = self.T1(x1, a1, alpha1)
            y[i][1] = self.T2(x1, x2, a2, alpha2)

        return y

    # ------------------------------------------------------------------

    def log_det_jacobian(self, x, theta):
        """ log |det J| of the transport map """

        _, alpha1, _, alpha2 = self.unpack(theta)

        return np.array([
            self.p1(x1, alpha1) + self.p2(x1, x2, alpha2)
            for x1, x2 in x
        ])

    # ------------------------------------------------------------------

```

```python
    def inverse(self, y, theta, max_iter = 100):
        """ finding the inverse """

        a1, alpha1, a2, alpha2 = self.unpack(theta)

        y = np.asarray(y, dtype=float)
        if y.ndim == 1:
            y = y.reshape(1, 2)

        n = y.shape[0]
        x = np.zeros((n, 2))

        for k in range(n):
            y1, y2 = y[k]

            # invert T1 by bisection
            lo, hi = -10.0, 10.0
            for _ in range(max_iter):
                mid = 0.5 * (lo + hi)
                if self.T1(mid, a1, alpha1) < y1:
                    lo = mid
                else:
                    hi = mid
            x1 = 0.5 * (lo + hi)

            # invert T2 by bisection (x1 is now fixed)
            lo, hi = -10.0, 10.0
            for _ in range(max_iter):
                mid = 0.5 * (lo + hi)
                if self.T2(x1, mid, a2, alpha2) < y2:
                    lo = mid
                else:
                    hi = mid
            x2 = 0.5 * (lo + hi)

            x[k, 0] = x1
            x[k, 1] = x2

        return x if n > 1 else x[0]

    # ----------------------------------------------------------------

    def safe_exp(self, u, clip=2000.0):
        """ required techincality to avoid overflow issues during training of the maps """

        return np.exp(np.clip(u, -clip, clip))
```

Listing 4: maps

```python
def tmis(n_iter, T, theta, log_posterior, seed = 12):
    """ independence sampler with the transport map in the proposal """

    # random seed
    rng = np.random.default_rng(seed)

    # starting prior sample
    eta = np.zeros(2, dtype = float).reshape((1, 2))

    # pushed sample
    x = T.forward(eta, theta)

    chain = [x[0].copy()] * n_iter
    acc_count = 0

    for i in range(1, n_iter):

        # acceptance probability
        u = rng.uniform()

        # proposal
        etap = rng.standard_normal(size = (1, 2))
        xp = T.forward(etap, theta)

        log_fY = log_posterior(xp)[0]
        log_fX = log_posterior(x)[0]

        log_gY = log_prior(etap)[0] - T.log_det_jacobian(etap, theta)[0]
        log_gX = log_prior(eta)[0]  - T.log_det_jacobian(eta,  theta)[0]

        log_acc = (log_fY - log_fX) + (log_gX - log_gY)

        if log_acc >= 0:
            acc = 1.0

        else:
            acc = np.exp(log_acc)

        if u < acc:
            x = xp
            eta = etap
            acc_count += 1

        chain[i] = x[0].copy()

    return np.array(chain), acc_count / n_iter
```

Listing 5: independence sampler

```python
def tmis_rwm(n_iter, T, theta, log_posterior, gamma = 0.5, step = 0.18, seed = 12):
    """ select tmis with probability gamma and rwm with probability 1 - gamma """

    # random numbers generator
    rng = np.random.default_rng(seed)

    # prior and pushed initial sample
    eta = np.zeros(2, dtype = float).reshape((1, 2))
    x = T.forward(eta, theta)

    chain = [x[0].copy()] * n_iter
    acc_count_rwm = 0
    acc_count_is = 0
    tmis = 0

    for i in range(1, n_iter):

        u1 = rng.uniform()
        u2 = rng.uniform()

        # with probability gamma do tmis
        if u1 <= gamma:
            tmis += 1

            etap = rng.standard_normal(size = (1, 2))
            xp = T.forward(etap, theta)

            log_fY = log_posterior(xp)[0]
            log_fX = log_posterior(x)[0]

            log_gY = log_prior(etap)[0] - T.log_det_jacobian(etap, theta)[0]
            log_gX = log_prior(eta)[0]  - T.log_det_jacobian(eta,  theta)[0]

            log_acc = (log_fY - log_fX) + (log_gX - log_gY)

            if log_acc >= 0:
                acc = 1.0

            else:
                acc = np.exp(log_acc)

            if u2 < acc:
                x = xp
                eta = etap
                acc_count_is += 1

        # with probability 1 - gamma do rwm
        else:

            xp = x[0] + step * rng.standard_normal(size = 2)
            xp = xp.reshape(1, 2)

            log_fY = log_posterior(xp)[0]
            log_fX = log_posterior(x)[0]

            log_acc = log_fY - log_fX

```

```
            if log_acc >= 0:
                acc = 1.0

            else:
                acc = np.exp(log_acc)

            if u2 < acc:
                x = xp
                eta = np.array(T.inverse(x[0], theta), dtype = float).reshape(1, 2)

                acc_count_rwm += 1

        chain[i] = x[0].copy()

    return np.array(chain), acc_count_is / tmis, acc_count_rwm / (n_iter - tmis)
```

Listing 6: mixed rwm + is

```python
def mh_inverse(n_iter, S, thetaS, log_posterior, sigma = 0.2, seed = 12):
    """ inverse map-based metroplis hastings algorithm """

    rng = np.random.default_rng(seed)

    # starting point of the chain
    x = np.zeros(2, dtype = float).reshape((1, 2))
    chain = [x[0].copy()] * n_iter
    acc_count = 0

    # depushed starting point
    xhat = S.forward(x, thetaS)

    for i in range(n_iter):

        u = rng.uniform()

        # prior-space proposal
        yhat = xhat + sigma * rng.normal(size=(1, 2))

        y = np.array(S.inverse(yhat[0], thetaS), dtype=float).reshape(1, 2)

        # log-acceptance rate
        log_acc = (log_posterior(y)[0] - log_posterior(x)[0] - S.log_det_jacobian(
            y, thetaS)[0] + S.log_det_jacobian(x, thetaS)[0])

        if log_acc >= 0:
            acc = 1.0

        else:
            acc = np.exp(log_acc)

        if u < acc:
            x = y
            xhat = yhat
            acc_count += 1

        chain[i] = x[0].copy()

    return np.array(chain), acc_count / n_iter
```

Listing 7: inverse maps algorithm

```python
def mh_inverse_adaptive(n_iter, S, thetaS, log_posterior, log_prior, sigma=0.2, adapt_every=5000, maxiter=100, seed=12):
    """ adaptive mh with the inverse map instead of the original one """

    rng = np.random.default_rng(seed)

    x = np.zeros(2, dtype=float).reshape((1, 2))
    chain = np.empty((n_iter, 2), dtype=float)
    acc_count = 0

    xhat = S.forward(x, thetaS)

    for i in range(n_iter):

        if i > 0 and adapt_every > 0 and (i % adapt_every == 0):
            Z = chain[:i].copy()

            def neg_log_KL(theta):
                Y = S.forward(Z, theta)
                log_phi = log_prior(Y)
                log_det = S.log_det_jacobian(Z, theta)
                return np.mean(-log_phi - log_det)

            res = opt.minimize(neg_log_KL, thetaS, method="L-BFGS-B", options={"maxiter": maxiter})
            thetaS = res.x
            xhat = S.forward(x, thetaS)

        u = rng.uniform()
        yhat = xhat + sigma * rng.normal(size=(1, 2))
        y = np.array(S.inverse(yhat[0], thetaS), dtype=float).reshape(1, 2)

        log_acc = (log_posterior(y)[0] - log_posterior(x)[0]) - (S.log_det_jacobian(y, thetaS)[0] - S.log_det_jacobian(x, thetaS)[0])

        if log_acc >= 0:
            acc = 1.0

        else:
            acc = np.exp(log_acc)

        if u < acc:
            x = y
            xhat = yhat
            acc_count += 1

        chain[i] = x[0].copy()

    return chain, acc_count / n_iter, thetaS
```

Listing 8: adaptive algorithm